

SUMMARY

=====

Retry → handle temporary failures
Offset → track consumer progress
DLQ → store permanently failed messages
Event ID → uniquely identify event
Duplicate handling → ensure idempotent processing

=====

KAFKA DLQ (Dead Letter Queue) - SHORT NOTE

=====

1) WHEN DOES A MESSAGE GO TO DLQ?

A message is sent to DLQ when the consumer fails to process it after retries.

Common reasons:

- Validation failure (invalid payload)
- Deserialization error (JSON parsing issue)
- Business rule failure (missing required fields)
- External dependency failure (DB/API down)
- Retry attempts exhausted

2) HOW DOES IT GO TO DLQ? (SPRING KAFKA)

Spring Kafka uses:

- DefaultErrorHandler
- DeadLetterPublishingRecoverer

Flow:

Consumer → Retry (e.g., 3 times) → Still fails → Sent to DLQ topic

Example config:

@Bean

```
public DefaultErrorHandler errorHandler(KafkaTemplate<String, Object> template) {  
    DeadLetterPublishingRecoverer recoverer =  
        new DeadLetterPublishingRecoverer(template);  
  
    DefaultErrorHandler handler =  
        new DefaultErrorHandler(recoverer, new FixedBackOff(1000L, 3));  
    return handler;  
}
```

```
}
```

3) EXAMPLE SCENARIO

Message:

```
{  
  "auditId": null  
}
```

Consumer logic:

if (auditId == null) throw Exception

Flow:

main-topic → consumer ❌ → retry → fail → DLQ topic

4) HOW TO FETCH MESSAGE FROM DLQ?

Option 1: Kafka Consumer

```
@KafkaListener(topics = "main-topic.DLT")  
public void readDlq(String message) {  
    System.out.println("DLQ message: " + message);  
}
```

Option 2:

- Kafka UI tools
- CLI consumer

5) HOW TO REPLAY A MESSAGE FROM DLQ?

Replay = produce message again to main topic

```
@KafkaListener(topics = "main-topic.DLT")  
public void replay(String message,  
    @Header(KafkaHeaders.RECEIVED_PARTITION_ID) int partition,  
    @Header(KafkaHeaders.RECEIVED_KEY) String key) {  
  
    kafkaTemplate.send("main-topic", key, message);  
}
```

6) PARTITION HANDLING DURING REPLAY

Case 1: No partition specified

→ Kafka assigns:

- Based on key (hashing)
- Or round-robin

Case 2: Preserve original partition (BEST PRACTICE)

```
kafkaTemplate.send(  
    new ProducerRecord<>("main-topic", partition, key, message)  
);
```

→ Message goes to SAME partition

7) OFFSET HANDLING

- Original offset is NOT reused
- Replay creates NEW message

Example:

Before:

Partition 0 → Offset 10 (failed)

After replay:

Partition 0 → Offset 25 (new message)

8) HOW TO HANDLE FAILED MESSAGE PROPERLY

Best practices:

1. Idempotency

- Use unique eventId
- Avoid duplicate processing

2. Retry limit

- Prevent infinite loops

3. Logging

- Store error details

4. DLQ metadata

- original topic, partition, offset

5. Alerting

- Monitor DLQ size

9) FINAL FLOW SUMMARY

Producer → Main Topic → Consumer

→ (Fail) → Retry → Fail

→ DLQ Topic

→ Ops Fix / Auto Replay

→ Produce Again → Main Topic

→ Consumer (Success)

KEY POINT:

Replay ≠ same message retry

Replay = NEW message with same data

##End to End flow

Audit Service

↓ (produces)

audit-events

↓

Rule Engine (consumer)

↓ (produces)

rule-events

↓

Notification Service (consumer)

How do you handle messages in Dead Letter Queue (DLQ) in your system?

We don't directly modify messages in Kafka. When a message goes to DLQ, we store it in a database and expose it via an Admin UI used by Ops.

Ops can review the failed payload, make necessary corrections, and trigger a retry.

The UI calls a backend API, which validates the data and republishes the message to the original Kafka topic.

We also maintain retry count and status to avoid infinite retries.

In non-production, we usually handle DLQ messages using Postman. Once we identify the issue and fix the data or code, we hit a retry API via Postman, which republishes the message to the original Kafka topic. Since it's non-prod, we don't rely on a full Admin UI and keep the process simple for faster debugging.

In our audit system, we use Kafka to publish audit events like finding creation. Multiple services like notification, dashboard, and rule engine consume these events asynchronously. We implemented retry with backoff and DLQ handling to ensure fault tolerance and allow manual replay of failed events.

◆ Who consumes the topic & why

- **Notification Service**

- Consumes: `audit.finding.created`
 - Purpose: Send email / in-app alerts to auditors, managers
 - Why async: Notification failure should not block audit creation
-

- **Dashboard Service**

- Consumes: `audit.finding.created`
 - Purpose: Update real-time metrics (open findings, severity count)
 - Why async: UI/dashboard updates can happen eventually (not critical path)
-

- **Rule Engine Service**

- Consumes: `audit.finding.created`
- Purpose: Evaluate dynamic rules (e.g., HIGH severity → escalate)
- Why async: Rule processing can be heavy and should be decoupled

-
- **Audit History / Logging Service (optional but realistic)**
 - Consumes: `audit.finding.created`
 - Purpose: Maintain audit trail / event history
 - Why async: Logging should not impact main transaction

◆ **Kafka Message Failure Summary**

- **Email sending failed**
 - SMTP server down / timeout / invalid email config
- **Database operation failed**
 - DB connection issue / deadlock / query timeout
- **Invalid event data**
 - Missing or null fields (e.g., `severity = null`) causing runtime exceptions
- **Code-level exception**
 - Bug in consumer logic (null pointer, divide by zero, etc.)
- **Downstream service failure**
 - Dependent microservice API is down or not responding
- **Schema mismatch / event format change**
 - Producer and consumer not aligned on event structure
- **Processing timeout / long execution**
 - Heavy logic causing consumer timeout or rebalance interruption

◆ **Key Understanding (important)**

- **Retry works for:** temporary issues (DB down, API failure)
- **DLQ needed for:** permanent issues (bad data, code bug, schema mismatch)

CI/CD

Docker

Kubernetes

Local Development

MICROSERVICE LOCAL RUN SETUP (SHORT NOTES)

PROJECT STRUCTURE

```
-----  
repo/  
├── services/  
│   ├── user-service/  
│   ├── order-service/  
│   └── payment-service/  
├── k8s/  
│   ├── user-service/  
│   ├── order-service/  
│   └── payment-service/  
└── start-local.ps1
```

WHAT HAPPENS AFTER GIT PULL

- ```

```
1. Start local Kubernetes cluster  
minikube start
  2. Use Minikube docker environment  
minikube docker-env
  3. Build docker images for services
  4. Deploy Kubernetes manifests  
kubectl apply -f k8s/
  5. Services communicate using Kubernetes DNS  
Example:  
http://user-service:8080

### OPTION 1 — RUN ALL SERVICES

```

```

Command:

```
.\start-local.ps1
or
.\start-local.ps1 all
```

What happens:

- Script scans "services/" directory
- Builds docker image for each service

- Deploys all k8s manifests
- Kubernetes creates pods and services

Example running pods:

user-service-pod  
order-service-pod  
payment-service-pod

Inter-service communication:

order-service → http://user-service:8080

## OPTION 2 — RUN SINGLE MICROSERVICE

---

Command:

.\start-local.ps1 order-service

What happens:

- Only order-service image is built
- Only order-service k8s manifest applied
- Other services are assumed to exist in cluster  
OR run in remote dev environment

Example flow:

order-service (local pod)  
↓  
http://user-service:8080  
↓  
user-service pod

## SERVICE COMMUNICATION

---

Services never call IPs.

They use the Kubernetes service name.

Example:

http://user-service:8080  
http://payment-service:8080

Kubernetes DNS automatically resolves.



## PORT MANAGEMENT

-----

Inside container:

`server.port=8080`

Kubernetes service mapping:

`port: 8080`

`targetPort: 8080`

Flow:

order-service pod



user-service service



user-service pod

## DEVELOPER WORKFLOW

-----

Fresh pull:

`git pull`

`.\start-local.ps1 order-service`

or

`git pull`

`.\start-local.ps1 all`

## KEY IDEA

-----

Script supports two modes:

- 1) Run full microservice system
- 2) Run only the service you are developing

DTO DAO

## STUDENT APP – SHORT NOTES (SPRING BOOT STRUCTURE)

### 1. PROJECT STRUCTURE

Typical layered architecture used in Spring Boot applications.

controller → Handles HTTP requests

service → Business logic

repository → Database interaction

entity → JPA entities mapped to DB tables

dto → Objects used for API request/response

mapper → Converts Entity ↔ DTO

Example package structure:

```
com.studentapp
├── controller
│ └── StudentController
├── service
│ ├── StudentService (interface)
│ └── StudentServiceImpl
├── repository
│ └── StudentRepository
├── entity
│ └── Student
├── dto
│ ├── StudentPersonalDTO
│ └── StudentCourseDTO
├── mapper
└── StudentMapper
```

---

### 2. ENTITY CLASS

Represents the database table.

```
@Entity
@Table(name="students")
class Student {
```

```
@Id
@GeneratedValue
private Long id;
```

```
private String name;
private String email;
private String schoolName;
```

```
private List<String> courses;
```

```
}
```

---

### 3. DTO CLASSES

DTO is used to send only required data to clients instead of exposing entities.

StudentPersonalDTO

```
class StudentPersonalDTO {
```

```
 private Long id;
 private String name;
 private String email;
```

```
}
```

StudentCourseDTO

```
class StudentCourseDTO {
```

```
 private String schoolName;
 private List<String> courses;
```

```
}
```

---

### 4. REPOSITORY (DAO)

Handles database operations.

@Repository

```
interface StudentRepository extends JpaRepository<Student, Long> {
```

```
}
```

---

### 5. SERVICE INTERFACE

Defines business operations.

```
interface StudentService {
```

```
StudentPersonalDTO getStudentPersonal(Long id);

StudentCourseDTO getStudentCourses(Long id);

}
```

---

## 6. SERVICE IMPLEMENTATION

Mapping using Java Streams.

```
@Service
class StudentServiceImpl implements StudentService {

 private StudentRepository repo;

 public StudentPersonalDTO getStudentPersonal(Long id) {

 Student student = repo.findById(id).orElseThrow();

 return List.of(student)
 .stream()
 .map(s -> new StudentPersonalDTO(
 s.getId(),
 s.getName(),
 s.getEmail()
))
 .findFirst()
 .orElse(null);
 }

 public StudentCourseDTO getStudentCourses(Long id) {

 Student student = repo.findById(id).orElseThrow();

 return List.of(student)
 .stream()
 .map(s -> new StudentCourseDTO(
 s.getSchoolName(),
 s.getCourses()
))
 .findFirst()
 .orElse(null);
 }

}
```

---

## 7. CONTROLLER

Exposes REST APIs.

```
@RestController
@RequestMapping("/students")
class StudentController {

 private StudentService service;

 @GetMapping("/personal/{id}")
 public StudentPersonalDTO getPersonal(@PathVariable Long id) {
 return service.getStudentPersonal(id);
 }

 @GetMapping("/courses/{id}")
 public StudentCourseDTO getCourses(@PathVariable Long id) {
 return service.getStudentCourses(id);
 }

}
```

---

## 8. WHY WE USE STREAM MAPPING

Instead of manual loops, Java Streams provide a functional and cleaner way.

Traditional approach

```
for(Student s : students){
 dtoList.add(convertToDTO(s));
}
```

Stream approach

```
students.stream()
 .map(StudentMapper::toDTO)
 .collect(Collectors.toList());
```

Advantages

- Cleaner and more readable code
- Functional programming style
- Easy transformation (Entity → DTO)
- Supports parallel processing

- Reduces boilerplate loops
  - Commonly used in microservice applications
- 

## 9. REAL PROJECT FLOW

Client → Controller → Service → Repository → Database

Response Flow

Database → Entity → Service → DTO Mapping → Controller → Client

---

## 10. WHY DTO IS USED

- Avoid exposing entity directly
- Control API response structure
- Improve security
- Reduce unnecessary data transfer
- Maintain separation between DB and API layer

# Circuit Breaker



```
=====
Circuit breaker config
=====
```

```
=====
RESILIENCE4J CIRCUIT BREAKER - SETUP NOTES
=====
```

## PURPOSE

-----

Circuit Breaker protects an application from repeatedly calling a failing service. If a service fails frequently, the circuit opens and prevents further calls, redirecting requests to a fallback method.

Example:

Primary Service : Redis

Fallback Service: Database

```

1. ADD DEPENDENCY (MAVEN)

```

```
<dependency>
 <groupId>org.springframework.cloud</groupId>
 <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>
```

```

2. ENABLE ACTUATOR (FOR MONITORING)

```

```
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

```

3. CONTROLLER IMPLEMENTATION

```

```
package in.ashokit.rest;
```

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;
```

```

@RestController
public class DemoRestController {

 @GetMapping("/data")
 @CircuitBreaker(
 name = "ashokit",
 fallbackMethod = "getDataFromDB"
)
 public String getDataFromRedis() {

 System.out.println("Calling Redis Service...");

 // Simulating failure
 int i = 10 / 0;

 return "Data fetched from Redis";
 }

 // FALLBACK METHOD
 // Must have same return type
 // Must accept Throwable parameter

 public String getDataFromDB(Throwable t) {

 System.out.println("Fallback triggered: calling DB");

 return "Data fetched from Database";
 }
}

```

---

#### 4. APPLICATION.YML CONFIGURATION

---

```

server:
 port: 8081

spring:
 application:
 name: resilience4j-demo

management:
 endpoints:
 web:

```

exposure:  
include: '\*'

endpoint:  
health:  
show-details: always

health:  
circuitbreakers:  
enabled: true

resilience4j:  
circuitbreaker:  
configs:  
default:  
  
registerHealthIndicator: true  
  
slidingWindowSize: 10  
minimumNumberOfCalls: 5  
  
permittedNumberOfCallsInHalfOpenState: 3  
  
automaticTransitionFromOpenToHalfOpenEnabled: true  
  
waitDurationInOpenState: 30s  
  
failureRateThreshold: 50  
  
eventConsumerBufferSize: 10

---

## 5. IMPORTANT CONFIGURATION PARAMETERS

---

slidingWindowSize

Number of requests used to calculate failure rate.

Example:  
slidingWindowSize = 10  
Circuit breaker tracks last 10 calls.

minimumNumberOfCalls